

Partie 03 — Sélectionner et transformer des données

INF-STAR — Programmation statistique avec R

Oskar Laverny

September 28, 2026

Aix-Marseille Université

Objectifs

À la fin de cette partie

Vous saurez :

- écrire et combiner des conditions logiques ;
- utiliser `if` / `else` pour une décision unique ;
- utiliser le pipe natif `|>` ;
- filtrer, trier, sélectionner et créer des variables avec `dplyr` ;
- calculer des résumés par groupe.

Logique



Comparaisons

```
age <- 67
```

```
age >= 65
```

```
age == 67
```

```
age != 50
```

Une comparaison retourne TRUE ou FALSE.

Combiner des conditions

```
age >= 65 & age < 80  
age < 18 | age > 85  
!(age >= 65)
```

- & : ET ;
- | : OU ;
- ! : NON.

Plusieurs valeurs à la fois

```
age <- c(34, 51, 68, 42)  
age >= 50
```

La comparaison est vectorisée : elle produit un booléen par élément.

```
age[age >= 50]
```

if et else



Une décision unique

```
age <- 17

if (age >= 18) {
  message("majeur")
} else {
  message("mineur")
}
```

if attend une seule condition TRUE ou FALSE.

Plusieurs cas

```
imc <- 27

if (imc < 18.5) {
  categorie <- "insuffisance pondérale"
} else if (imc < 25) {
  categorie <- "corpulence normale"
} else {
  categorie <- "surpoids ou obésité"
}
```

if n'est pas une boucle sur les lignes

Pour un vecteur entier, on utilise une opération vectorisée :

```
age <- c(12, 20, 70)
ifelse(age >= 18, "majeur", "mineur")
```

Ou, dans dplyr :

```
dplyr::if_else(age >= 18, "majeur", "mineur")
```

Le pipe |>



Lire le code de gauche à droite

Sans pipe :

```
round(mean(c(12.1, 15.7, 18.4)), 1)
```

Avec pipe :

```
c(12.1, 15.7, 18.4) |>  
  mean() |>  
  round(1)
```

Le pipe natif `|>` est disponible depuis R 4.1.

Transformer avec dplyr



```
library(dplyr)
```

Nous utiliserons quelques verbes principaux :

- `filter()` : choisir des lignes ;
- `select()` : choisir des colonnes ;
- `arrange()` : trier ;
- `mutate()` : créer/modifier des colonnes ;
- `summarise()` : résumer ;
- `group_by()` : travailler par groupe.

filter()

```
patients |>  
  filter(age >= 65)
```

Plusieurs critères :

```
patients |>  
  filter(age >= 65, fumeur == "oui")
```

Valeurs manquantes et filtrage

```
patients |>  
  filter(!is.na(tas))
```

Une comparaison avec **NA** ne vaut ni **TRUE** ni **FALSE** : elle reste indéterminée.

D'où l'importance de `is.na()`.

```
select()
```

```
patients |>  
  select(id, age, sexe, tas)
```

On peut aussi exclure :

```
patients |>  
  select(-date_visite)
```

```
patients |>  
  arrange(age)
```

Ordre décroissant :

```
patients |>  
  arrange(desc(age))
```

```
patients |>
  mutate(
    taille_m = taille_cm / 100,
    imc = poids_kg / taille_m^2
  )
```

Une nouvelle variable peut utiliser une variable créée juste avant dans le même `mutate()`.

Catégoriser avec `case_when()`

```
patients |>
  mutate(
    classe_age = case_when(
      age < 40 ~ "< 40",
      age < 65 ~ "40-64",
      TRUE ~ "65+"
    )
  )
```

`TRUE` joue ici le rôle du « sinon » final.

Résumer



```
patients |>
  summarise(
    n = n(),
    age_moyen = mean(age),
    tas_moyenne = mean(tas, na.rm = TRUE)
  )
```

Résumer par groupe

```
patients |>
  group_by(sexe) |>
  summarise(
    n = n(),
    age_moyen = mean(age),
    .groups = "drop"
  )
```

Le regroupement change la signification de `summarise()` : le calcul est répété pour chaque groupe.

Un workflow complet

```
patients |>
  filter(!is.na(poids_kg)) |>
  mutate(imc = poids_kg / (taille_cm / 100)^2) |>
  group_by(sexe) |>
  summarise(
    n = n(),
    imc_moyen = mean(imc),
    .groups = "drop"
  ) |>
  arrange(desc(imc_moyen))
```

Base R et dplyr

Deux syntaxes valides

Base R :

```
patients[patients$age >= 65, c("id", "age", "tas")]
```

dplyr :

```
patients |>  
  filter(age >= 65) |>  
  select(id, age, tas)
```

Le cours utilise les deux : comprendre l'indexation base R aide à comprendre le langage ; `dplyr` est souvent plus lisible pour les analyses de données.

À retenir



Un petit vocabulaire très puissant

```
condition -> filter -> select -> mutate -> group_by -> summarise
```

Avec quelques verbes et le pipe `|>`, on peut déjà réaliser une grande partie des transformations courantes d'un tableau de données.